9️⃣ Mathematical Operations — Concept 1

🔹 Element-Wise Operations (Foundation of Everything)

🧠 1. What Are Element-Wise Operations?

Element-wise means:

Operation happens between corresponding elements of arrays.

If two arrays have the same shape:

```
A = [a1, a2, a3]
B = [b1, b2, b3]
```

Then:

```
A + B = [a1+b1, a2+b2, a3+b3]
```

Each position operates independently.

🧮 2. Basic Element-Wise Operators

NumPy supports:

Operation	Symbol	Function Version
Addition	`+`	`np.add()`
Subtraction	`-`	`np.subtract()`
Multiplication	`*`	`np.multiply()`
Division	`/`	`np.divide()`
Power	`**`	`np.power()`
Modulus	`%`	`np.mod()`

Important:

`+` and `np.add()` are the SAME internally (ufunc — we'll cover that next).

🧪 3. Code Examples

```
import numpy as np

a = np.array([10, 20, 30])
b = np.array([1, 2, 3])

print(a + b)
# [11 22 33]

print(a - b)
# [ 9 18 27]

print(a * b)
# [10 40 90]

print(a / b)
# [10. 10. 10.]

print(a ** 2)
# [100 400 900]
```

Notice:

- Operations are vectorized
- No loops
- Extremely fast

⚡ 4. Why This Is Important for ML & Data Analysis

In Data Analysis:

- Profit = revenue - cost
- Percentage growth = (new - old) / old
- Feature scaling = x - mean

In ML:

- Gradient descent updates
- Weight adjustments
- Error calculations
- Loss functions

Everything is element-wise math.

🚨 5. Important Rule

Element-wise operations require:

Same shape arrays
OR
Compatible shapes (broadcasting — next concept)

If shapes don't match:

```
a = np.array([1,2,3])
b = np.array([1,2])

print(a + b)
```

This gives:

```
ValueError: operands could not be broadcast together
```

We'll solve this in broadcasting section.

🧠 6. Element-Wise vs Matrix Multiplication (CRITICAL)

Many beginners confuse:

```
a * b   → element-wise
a @ b   → matrix multiplication
```

Example:

```
a = np.array([1,2,3])
b = np.array([4,5,6])

print(a * b)    # element-wise
print(a @ b)    # dot product
```

Output:

```
[ 4 10 18]
32
```

That 32 = 1×4 + 2×5 + 3×6

Big difference.

This confusion destroys ML understanding if not clear.

🔬 7. Performance Insight (Professional Level)

Element-wise operations are:

- Implemented in C
- Use contiguous memory
- Avoid Python loops
- SIMD optimized internally

That's why:

NumPy is much faster than lists.

🔹 2. Universal Functions (ufuncs)

🧠 1. What Are ufuncs?

Universal functions (ufuncs) are **fast, element-wise functions** implemented in C inside NumPy.

They operate on entire arrays **without explicit loops**.

Think of them as **vectorized math functions**.

Examples of ufuncs:

- Arithmetic: `np.add`, `np.subtract`, `np.multiply`, `np.divide`, `np.power`
- Trigonometric: `np.sin`, `np.cos`, `np.tan`
- Exponential / Log: `np.exp`, `np.log`, `np.sqrt`
- Statistical: `np.maximum`, `np.minimum`

Key point:

```
a = np.array([1,2,3])
b = np.array([4,5,6])
```

```
np.add(a, b)    # element-wise addition
```

Is **exactly equivalent** to:

```
a + b
```

🧮 2. Why Use ufuncs?

- They are **vectorized** → extremely fast
- Avoid Python loops → memory efficient
- Can be **broadcasted** across arrays of different shapes
- Support **optional arguments** (`out=`, `where=`)

🧪 3. Code Examples

```
import numpy as np

a = np.array([1,2,3])
b = np.array([4,5,6])

# Arithmetic
print(np.add(a, b))        # [5 7 9]
print(np.subtract(a, b))   # [-3 -3 -3]
print(np.multiply(a, b))   # [ 4 10 18]
print(np.divide(b, a))     # [4. 2.5 2.]

# Power / Square root
print(np.power(a, 3))      # [ 1 8 27]
print(np.sqrt(b))           # [2. 2.236 2.449]

# Trigonometric
angles = np.array([0, np.pi/2, np.pi])
print(np.sin(angles))      # [0. 1. 0.]
print(np.cos(angles))      # [1. 0. -1.]
```

Notice how **all operations are element-wise and super fast**.

⚡ 4. Advanced Features of ufuncs

4.1 `out` parameter

Store results directly in an array → avoids creating new memory:

```
result = np.empty(3)
np.add(a, b, out=result)
print(result)   # [5 7 9]
```

4.2 `where` parameter

Conditionally apply function:

```
np.add(a, b, where=[True, False, True])
# Only adds elements where True
```

4.3 Unary ufuncs

- Operate on **single array**
- Examples: `np.sqrt(a)`, `np.exp(a)`, `np.abs(a)`

🔬 5. Why ufuncs Matter for ML & Data Analysis

Data Analysis:

- Compute revenue growth: `np.subtract(new, old)`
- Normalize data: `np.divide(x - mean, std)`
- Apply conditional transformations: `np.where(condition, x, y)`

ML:

- Activation functions: `np.tanh`, `np.exp`
- Loss computation: element-wise subtraction / square → `np.power`
- Vectorized gradient computation

💡 Any large dataset requires ufunc thinking for speed and correctness.

oncept 3 — Broadcasting (VERY IMPORTANT)

This is **the cornerstone of professional NumPy**, ML, and data analysis.

Why broadcasting is next:

- Element-wise operations **require compatible shapes**
- But most real datasets are **different shapes** (e.g., 1D feature array + 2D data matrix)
- Broadcasting **lets NumPy "stretch" arrays without copying data**
- It makes vectorized math **practical and fast** for ML

What we'll cover in this part:

1. Broadcasting rules (the 3-step mental model)
2. Practical examples (1D → 2D, scalars → arrays)
3. Common pitfalls & errors
4. How arithmetic operators and ufuncs behave with broadcasting

Broadcasting — Step 1: Scalar with Array

This is the simplest case.

1️⃣ Concept

When you do:

```
import numpy as np

a = np.array([1, 2, 3])
b = 5
```

If you do:

```
a + b
```

NumPy **"stretches" the scalar** `b` to match the shape of `a` internally:

- `b` becomes `[5, 5, 5]` in memory (conceptually, not really copying data)
- Then element-wise addition happens: `[1+5, 2+5, 3+5]` → `[6, 7, 8]`

✅ Result: `[6, 7, 8]`

This is **broadcasting**.

2️⃣ Code Example

```
a = np.array([10, 20, 30])
scalar = 2

# Arithmetic operator
print(a + scalar)     # [12 22 32]
```

```
# ufunc
print(np.add(a, scalar))    # [12 22 32]
```

Notice: **both + and np.add work identically** — broadcasting happens automatically.

3️⃣ Key Idea

- **Shape of scalar:** `()`
- **Shape of array:** `(3,)`
- NumPy **stretches scalar** to match array

✅ Very powerful — this is exactly what happens when you normalize data, add bias in ML, or scale arrays.

🔹 Step 2: 1D Array with 1D Array (Different Lengths)

Broadcasting works only if:

1. Arrays have **same shape**
2. Or **one of the dimensions is 1**

Example:

```
a = np.array([1, 2, 3])
b = np.array([10])
```

`b` is shape `(1,)` — NumPy stretches it to `(3,)` → `[10, 10, 10]`

```
print(a + b)    # [11 12 13]
```

✅ Rule: NumPy compares shapes **from the right**, and a dimension is compatible if:

- Equal
- Or 1

Otherwise → ValueError.

4️⃣ Step 3: 2D Array with 1D Array (ML-style)

This is **very important for data analysis and ML**.

```
X = np.array([[1,2,3],
              [4,5,6],
              [7,8,9]])   # shape (3,3)

b = np.array([1,0,1])    # shape (3,)
```

If we do:

```
X + b
```

- NumPy aligns **from the right**
- `b` shape `(3,)` is compatible with each row `(3,)` → **broadcasted across rows**
- Result:

```
[[2, 2, 4],
 [5, 5, 7],
 [8, 8, 10]]
```

✅ This is **how you add bias to each feature** in ML.

🔬 Key Broadcasting Rules (Summary)

1. Compare shapes **from right to left**
2. A dimension is compatible if:
    - They are equal
    - Or one of them is 1
3. If shapes are incompatible → `ValueError`
4. Scalars are always compatible

Remaining Broadcasting Concepts

1. **Broadcasting with higher-dimensional arrays (3D, 4D, etc.)**
    - NumPy compares **from the rightmost axis**
    - Example: `(3,4,5) + (5,)` → last dimension matches → broadcast across first two axes
    - Example: `(3,4,5) + (1,4,1)` → shape (1,4,1) stretches across 3 and 5
2. **Using `np.newaxis` / `None`**
    - Allows you to **explicitly expand dimensions** for broadcasting
    - Example:

```
X = np.array([1,2,3])       # shape (3,)
X[:, np.newaxis]             # shape (3,1)
```

- This is super important for **ML when aligning vectors with matrices**

3. **Common pitfalls**
    - Mismatched shapes → `ValueError`
    - Thinking broadcasting **copies data** → it does **not**; it only creates a **view**

- Be careful with `where` + `out` → out must **match final broadcasted shape**

4. **Broadcasting in ML & Data Analysis**
   - Adding **bias term** to all samples
   - Scaling each feature across all samples
   - Vectorized gradient updates
   - Conditional operations without loops

Example: `(3,4,5) + (1,4,1)`

Step 1 — Understand the shapes

```
import numpy as np

A = np.zeros((3,4,5))    # shape (3,4,5)
B = np.ones((1,4,1))     # shape (1,4,1)
```

- `A.shape = (3,4,5)`
- `B.shape = (1,4,1)`

NumPy compares shapes **from the right**:

Axis	A	B	Compatible?
last (axis 2)	5	1	✅ (1 can stretch)
middle (axis 1)	4	4	✅ (equal)
first (axis 0)	3	1	✅ (1 can stretch)

✅ All axes are compatible → broadcasting works

Step 2 — How broadcasting stretches

- `B` expands **implicitly** to match `A.shape`:

```
B = [[[1,1,1,1,1],        <-- stretched along last axis
      [1,1,1,1,1],
      [1,1,1,1,1],
      [1,1,1,1,1]],

     [[1,1,1,1,1],        <-- stretched along first axis
      [1,1,1,1,1],
      [1,1,1,1,1],
      [1,1,1,1,1]],

     [[1,1,1,1,1],
      [1,1,1,1,1],
      [1,1,1,1,1],
      [1,1,1,1,1]]]
```

Then `A + B` is element-wise addition → shape `(3,4,5)`

✅ Step 3 — Practical check

```
C = A + B
print(C.shape)  # (3,4,5)
print(C[0,0,:]) # [1. 1. 1. 1. 1]  <-- first row, first layer
```

🔹 2️⃣ `np.newaxis` Example

`np.newaxis` lets you **add a dimension** to an array explicitly so broadcasting works perfectly.

```
X = np.array([1,2,3])         # shape (3,)
X2 = X[:, np.newaxis]          # shape (3,1)
```

Step 1 — Visualize shapes

```
Original X:        [1, 2, 3]      shape: (3,)
After [:, np.newaxis]:
[[1],
 [2],
 [3]]                shape: (3,1)
```

- Notice the **extra dimension** added as a column

Step 2 — Use broadcasting

```
Y = np.array([10,20,30])      # shape (3,)

# Add X2 + Y
Z = X2 + Y
print(Z)
```

Step 3 — Result

```
X2 (3,1) + Y (3,)  → broadcasting along columns
Z =
[[11 21 31],
 [12 22 32],
 [13 23 33]]
```

✅ How it worked:

- `X2` shape `(3,1)` → broadcast across last dimension
- `Y` shape `(3,)` → treated as `(1,3)` → broadcast across first dimension

So final shape = `(3,3)`

🔹 Key Takeaways

1. `np.newaxis` = add dimension so shapes **align for broadcasting**
2. Broadcasting is **implicit**, but `np.newaxis` gives you **explicit control**
3. Works with **any dimension** — 1D, 2D, 3D… very important for ML matrices

Custom ufuncs

A **custom ufunc** is basically a **user-defined function that operates element-wise on NumPy arrays**, just like `np.add`, `np.sin`, or `np.sqrt`.

Why it matters:

- Avoids **slow Python loops**
- Works seamlessly on **arrays of any shape** (thanks to broadcasting)
- Lets you implement **activation functions, thresholds, transformations**
- Useful for **feature engineering and preprocessing** in ML

My Explanation:

A simple function written in Python by the programmer, he defines a python function consisting of programmer-desired functionalities and then he wants to implement that function on an array, so he vectorizes it by using np.vectorize

---

🔹 2️⃣ Basic Way: Using `np.vectorize`

`np.vectorize` takes a Python function and returns a **vectorized version** that applies the function **element-wise to arrays**.

Example 1 — Square Function

```
import numpy as np

# Define a simple Python function
def square(x):
    return x ** 2

# Vectorize it
vec_square = np.vectorize(square)
```

```
# Apply on array
arr = np.array([1, 2, 3, 4])
result = vec_square(arr)
print(result)  # [1 4 9 16]
```

✅ Key Points:

1. Works on **any shape array**
2. Handles **broadcasting** automatically
3. No need for loops

Example 2 — Conditional Function (ML style)

Suppose you want a **ReLU** (rectified linear unit) function:

```
def relu(x):
    return x if x > 0 else 0

# Vectorize
vec_relu = np.vectorize(relu)

arr = np.array([-2, -1, 0, 1, 2])
print(vec_relu(arr))  # [0 0 0 1 2]
```

💡 In ML, ReLU is used as an **activation function**.
Notice how we **directly applied it to the array** without loops.

- **Without vectorize:** you'd write a for loop:

```
arr = np.array([1,2,3,4])
result = []
for x in arr:
    result.append(square(x))
result = np.array(result)
```

- **With vectorize:** 1 line:

```
result = vec_square(arr)
```

✅ Cleaner, **broadcasting-aware**, easy to read

3️⃣ Using `np.frompyfunc` (Optional, Advanced)

- `np.frompyfunc` creates a **NumPy ufunc from a Python function**
- Works **element-wise**, and is a bit faster than `np.vectorize`

- Returns **object dtype** by default

Example:

```
def cube(x):
    return x ** 3

ufunc_cube = np.frompyfunc(cube, 1, 1)  # 1 input, 1 output
arr = np.array([1,2,3])
print(ufunc_cube(arr))  # [1 8 27] dtype=object
```

⚠️ Note: output is **object type**. For numeric operations, `np.vectorize` is often simpler.

🔹 4️⃣ Broadcasting with Custom ufuncs

Custom ufuncs **fully support broadcasting**:

```
def add_with_bias(x, b):
    return x + b

vec_add_bias = np.vectorize(add_with_bias)

X = np.array([[1,2,3],[4,5,6]])
bias = np.array([10, 20, 30])

print(vec_add_bias(X, bias))
# [[11 22 33]
#  [14 25 36]]
```

✅ Same broadcasting rules as built-in ufuncs

🔹 5️⃣ ML/Feature Engineering Examples

Example 1 — Sigmoid Function

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

vec_sigmoid = np.vectorize(sigmoid)

arr = np.array([-2, 0, 2])
print(vec_sigmoid(arr))  # [0.119 0.5 0.881]
```

Example 2 — Threshold Function

```
def threshold(x, t=0.5):
    return 1 if x > t else 0
```

```
vec_threshold = np.vectorize(threshold)

arr = np.array([0.2, 0.6, 0.4, 0.9])
print(vec_threshold(arr))  # [0 1 0 1]
```

🔹 6️⃣ Key Takeaways

1. `np.vectorize` = simplest way to create **element-wise functions**
2. Supports **any shape + broadcasting**
3. Use for:

- Activation functions
- Conditional transformations
- Feature scaling / normalization
- Custom mathematical operations

4. `np.frompyfunc` = alternative for slightly faster or more ufunc-like behavior, but returns **object type**
5. Avoid **Python loops** for large arrays — **vectorization is faster and memory-efficient**